CSE331 OPERATING SYSTEMS DESIGN FALL 2025 ASSIGNMENT I

Ece Düzgeç. 20220702045

Unix Processes And Posix Threads for Matrix Multiplication *Performance Comparison Report*

This report presents an analysis of four different approaches to matrix multiplication: multiprocess, multithreaded with mutual exclusion, multithreaded without mutual exclusion, and single process implementations. I tested four different ways to multiply matrices dimensions ranging from 10x10 to 800x800, and measured how fast each one runs. The goal was to understand when to use processes vs threads, and whether using locks (mutex) helps or hurts performance.

1. Multiprocess Program (hw1a.c)

This program creates separate processes similar to multiple programs running at once. Each process calculates one row of the answer.In the code main program creates N child processes (N = matrix size) Each child calculates one row independently and results stored in shared memory so all can see them while main program waits for all children to finish.

In this program

fork() : Creates a new process like cloning the program
shmget() : Creates shared memory space
wait() : Lets parent waits for children to finish
exit() : Ends it when child finishes its work

Pros of this program are that the processes are isolated so if one of them stops working, others keep running and also can work in parallel on different CPU cores. However creating processes is slow and because each process needs its own space, it uses lots of memory.

2. Multithreaded Program With Mutual Exclusion (hw1b.c)

It creates N threads where threads share the same memory and each thread calculates one row. The program also uses locks (mutex) before accessing matrices to prevent conflicts. As only one thread can access data at a time when locked.

In this program:

pthread_create() : Creates a new thread
pthread_mutex_lock() : Locks access so that other threads wait
pthread_mutex_unlock() : Unlocks so others can access
pthread_join() : Waits for thread to finish

This program has a major problem, it is very slow. Because threads spend most of their time waiting for locks and also only one thread works at a time. In this program where locks were added is to prevent conflicts when accessing shared data. *But, they're not needed here as* each thread writes to a different row, so they don't *actually* interfere with each other.

3. Multithreaded Program Without Mutual Exclusion (hw1c.c)

This program creates threads like the previous one, but it removes the locks instead. Creates N threads and each thread directly accesses and calculates its row. There is no locks so all threads work simultaneously and there is also no waiting and conflicts.

In this program:
pthread_create() : Creates threads
pthread_join() : Waits for threads
pthread_exit() : When thread finishes

This program is the best out of all of them for larger matrices because all threads work at the same time and there is no time wasted waiting for locks. Thread 0 only writes to row 0, thread 1 only writes to row 1, etc. They never write to the same place, so they can't interfere with each other. So we can see that without locks in matrix multiplication we can use all CPU cores effectively

4. Single Process Program (hw1d.c)

This program is just straightforward calculation. There are three nested loops calculate the entire result which processes rows one by one, sequentially**.** There are no processes, threads, or locks.

This program is the fastest for small matrices under 100x100. Because creating processes or threads takes time, and for small problems, that setup time is more than the calculation time.It has the simplest code and uses the least memory space. However, it becomes slow for large matrices and can't use multiple CPU cores

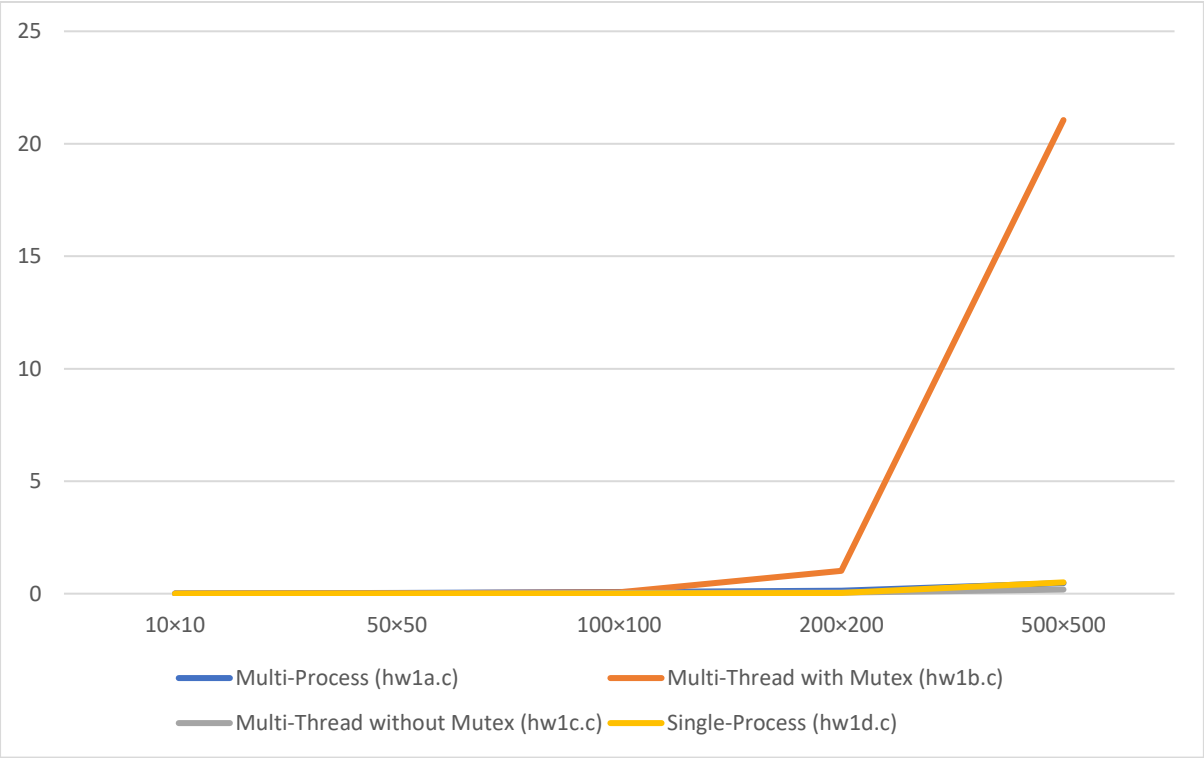
N (Dimension)	Multiprocess (hw1a.c)	Multithread with Mutex (hw1b.c)	Multithread without Mutex (hw1c.c)	Single Process (hw1d.c)
10×10	0.0096	0.0034	0.0044	0.0001
50×50	0.0372	0.0139	0.0107	0.0007
100×100	0.0703	0.0566	0.0259	0.0044
200×200	0.1366	1.0078	0.0538	0.0298
500×500	0.4686	21.0544	0.1832	0.5022

I tested each method. After testing how long each program took with different matrix sizes.

The results shows us that for small matrices such as 10×10 and 50×50 Single process is the best. As the calculations are so quick that the time to create processes or threads is longer than just doing the work directly

For medium matrices as 100×100,threads without locks is more effective as the calculation takes longer, so splitting among threads is better. We can see that threads version is about 6 times faster than single process here.

For large matrices such as 500×500 threads without locks is again the best out of all of them. We can see a clear difference between the times of the threads for larger matrices here as there is more and more values that needs calculation.



We can clearly see from the execution times of the programs that even though the locks were added to prevent conflicts, but since each thread works on a different row, so conflicts can't happen anyway. The locks just slow things down for no benefit.

So all in all, from these programs we can say that for small matrices under 100×100 as setup time is longer than the work time, Single Process works better. However, for larger matrices that are100×100 and up Threads Without Locks are more efficient and faster as there is no waiting. We can say that when there is a need for safety and isolation processes are worth slowing down. Then again that doesn’t mean that we can use threads With Locks under any situation as they can be unnecessary and slow down the performance.